



MCP
Dev Summit
Seoul

The 5 Wrong Reasons to Build an MCP Server (And What to Do Instead)

Daniel Oh

DEVELOPER ADVOCATE | JAVA CHAMPION | CNCF AMBASSADOR



@danieloh30



Developer Advocate at IBM (Red Hat)

- Cloud Native Runtimes
- AI, GitOps, Security, and Serverless



JAVA Champion



CNCF Ambassador & TAG DevEx co-chair



AAIF Ambassador



Microsoft MVP – Java & Developer Tools



Keynote Speaker & Published Author



Living in Boston, MA USA



Java[™]
Champions

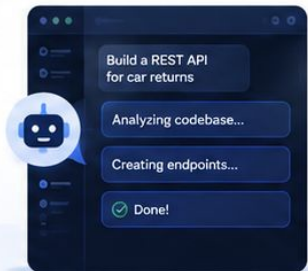


AI Landscape

In 2026, the demand for AI to 'do things' is at an all-time high, leading to **hasty decisions**.



The explosion of the “**Agentic IDE**” ecosystem.



New tools, new promises, every week.
FOMO is real.



5,000+ community MCP servers listed on central registries.



Incredible innovation, but varying **quality, security, and trust**.



The corporate pressure to “**AI-enable**” every internal API.



Every API is now a candidate for an **LLM wrapper**.



Why “**Standardized**” does not always mean “**Architecturally Sound**.”



A common interface does not guarantee a **good system**.



The risk of the “**Everything looks like a nail**” syndrome.



Not every problem is an **AI problem**. Not every API needs an **agent**.



Speed is tempting.
Architecture is **forever**.



Success comes from **intentional design**, not just **AI adoption**.

No is the Right Answer



MCP
Dev Summit
Seoul

WRONG REASON #1 (X)

Single Consumer Trap

If you control both the LLM client and the tool,
adding MCP is like **hiring a translator** for a conversation you're having with yourself.

OVER-ENGINEERED

Using MCP (Unnecessary)



Extra abstraction. Extra hops.
Extra latency. No real benefit.

RIGHT WAY

Direct Function Calling



Direct, simple, fast.
Exactly what you need.



Direct function calling avoids
the **"Middleman Tax."**



Reducing the **"Party of One"**
middleware overhead.



Maintaining a direct API is
2x faster than a wrapped one.



Complexity should only be added
to solve **distribution problems**.



One Agent + One Tool = **Zero** reason for a protocol.



Don't add a protocol
until you have a
distribution problem.



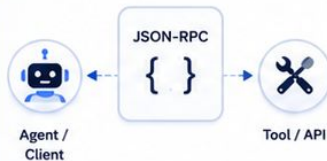
Performance Myth

Modern doesn't equal fast.

The transport dance in MCP can add **millisecond penalties** that break high-frequency loops.



JSON-RPC is a **convenience**, not a performance optimization.



Great for interoperability.
Not built for speed.



The hidden cost of **serialization** and **deserialization**.



- Encode (Object → JSON)
- Transmit
- Decode (JSON → Object)

Every hop adds overhead.
Every loop multiplies it.



Latency penalties in **Stdio** vs. **HTTP/SSE** transports.

STDIO



Fast local pipe, but blocking and fragile.

~10-30ms

HTTP / SSE

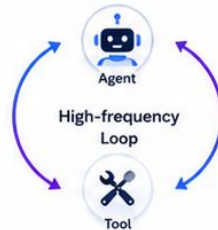


Network hops, headers, TLS, and queues.

~50-150ms+



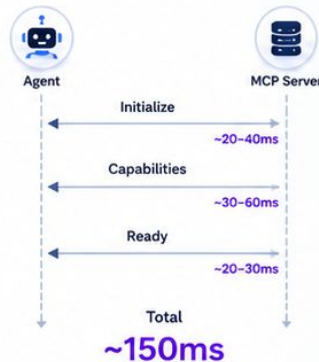
Why real-time **agentic loops** suffer under protocol overhead.



Overhead compounds. Loops slow down. Agents **underperform**.



Measuring the **150ms "Handshake" tax**.



Great protocols solve **distribution**. They don't make **local fast**.



Optimize for performance first. Add **protocols** only when **distribution** demands it.

Security Fallacy

Exposing raw tools to an LLM without a robust middle-tier is a recipe for a **data leak**. MCP doesn't fix your **auth story**.

1

MCP is a **transport layer**, not a security boundary.



MCP moves messages. It does **not verify identity or intent**.

2

The danger of **trusting** the LLM client blindly.



LLMs follow instructions. Malicious or compromised clients **can do damage**.

3

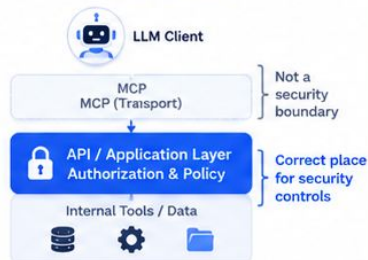
Increasing the **"Prompt Injection"** surface area via tools.



More tools = **more entry points** for prompt injection and malicious manipulation.

4

Why authorization belongs at the **API level**, not the MCP level.



Enforce authZ, rate limits, and policy at the **API layer** where context exists.

5

The **"Exposed Internal Tool"** nightmare scenario.



One misconfiguration. One exposed tool. **Massive breach.**



Security is a **design decision**, not a feature toggle.



Secure the API. **Govern** the tools. Don't outsource **trust** to a protocol.

Documentation Replacement



The LLM will still hallucinate if your underlying resource names are opaque.

Don't use MCP to hide technical debt.

1

MCP won't fix a poorly designed, undocumented API.



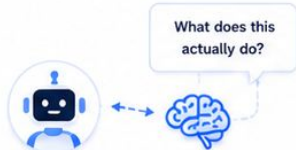
- ✗ Inconsistent naming
- ✗ Missing descriptions
- ✗ Unclear behavior



MCP is a transport. It **doesn't** make the API better.

2

LLMs still require high-quality semantic context to work.



Context drives correct understanding and results.

3

The JSON Schema is not "Auto-Intelligence."

```
1 {
2   "type": "object",
3   "properties": {
4     "customerId": {
5       "type": "string",
6       "description": "ID of the customer"
7     },
8     "startDate": {
9       "type": "string",
10      "format": "date",
11      "description": "Start date (YYYY-MM-DD)"
12    },
13   },
14   "required": ["customerId", "startDate"]
15 }
```



Schema describes shape, not meaning. It's not self-explanatory.

4

Why authorization belongs at the API level, not the MCP level.



Enforce authZ, rate limits, and policy at the API layer where context exists.

5

Descriptions are code: If you can't describe it, the LLM can't use it.

Poor Description



"Gets user data."
"Updates status."

Good Description



"Retrieves a user by ID."
Includes profile, preferences, and account status."



Clear, specific, and complete descriptions unlock LLM accuracy and reliability.



Documentation isn't optional. It's **part of your product**.



Fix the foundation. Then expose it **intelligently**.

Simple CRUD Operations



If the LLM can already use a browser or a shell, stay out of its way.

Don't build a bridge for a 2-foot gap.

1

HTTP verbs are already a "Standard Protocol."



✓ Battle-tested.
Widely supported.
No extra layer needed.

💡 You already have a standard.
Use it.

2

Why wrap a single GET request in a server manifest?



✗ More files. More layers.
Zero added value.

3

The maintenance burden of a server for one endpoint.



✗ Overhead that provides no business value.

4

When a cURL command or a shell script is the better "Tool."

cURL (Simple & Direct)

```

$ curl -X GET \
  'https://car-service.apps.local/q/api/cars/123' \
  -H 'Accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIs...'
  
```

Shell Script (Powerful & Flexible)

```

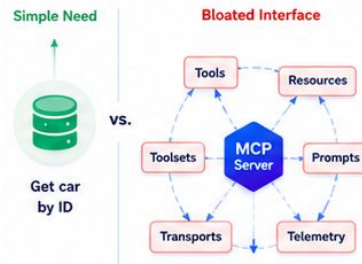
#!/bin/bash
set -euo pipefail
BASE_URL="https://car-service.apps.local/q/api"
ID="$1"

curl -s -H "Accept: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  "${BASE_URL}/cars/${ID}" | jq .
  
```

✓ Visible. Editable. Portable.
No extra infrastructure.

5

Avoiding "Interface Bloat" for basic data retrieval.



✗ Don't over-engineer what is already simple.



Use the **simplest tool** that works.

Add MCP when you have distribution, discoverability, or multi-consumer needs.



Don't build infrastructure for a 2-foot gap.

Architecture & Battle Scars



MCP
Dev Summit
Seoul



Mega Server Mistake

The result of our 'God Server' was an LLM that forgot how to use basic tools because it was **overwhelmed by options**.



1

The 50-tool manifest that broke the context window.

```

{
  "tools": [
    { "name": "get_user", ... },
    { "name": "create_user", ... },
    ...
    { "name": "export_legacy_report_v2", ... }
  ]
}
  
```

50 Tools

~18,400 tokens just for the manifest
(28% of a 64K context window)

2

Model "Decision Fatigue":
Picking the wrong tool
30% of the time.



Correct tool chosen
only 70% of the time

3

Token bloat: Sending
the whole schema
every single turn.

Every Turn



**~18.4K
tokens**

10 Turn
Conversation



**~184K
tokens**

**Wasted tokens = higher latency,
higher cost, lower throughput**

4

Why "Micro-Servers" are
usually better than
"God-Servers."

God Server



- ✗ One giant surface area
- ✗ Hard to maintain
- ✗ Confusing for models
- ✗ Higher error rate
- ✗ More tokens, more cost

Micro-Servers



- ✓ Focused responsibility
- ✓ Easy to maintain
- ✓ Clear for models
- ✓ Lower error rate
- ✓ Fewer tokens, lower cost

5

The importance of
Tool **Discovery** vs.
Tool **Overload**.



- ✓ Right tools, right time
- ✓ Context-aware discovery
- ✗ Too many choices
- ✗ Model confusion & errors

BEFORE: The "God Server" Anti-Pattern



VS.

AFTER: Micro-Servers + Smart Discovery



Don't give your LLM the entire toolbox. Give it a **flashlight**, not a **warehouse**.

Build micro-servers, enable smart discovery, and let the model focus on what matters.





STANDARDIZED ISN'T FREE

Token Efficiency Table

Use this data to justify why some tools should remain as simple APIs



Key Takeaway

Every token spent on tooling is a token not available for reasoning.

1. TOOL SCHEMA OVERHEAD: MCP vs. SIMPLE FUNCTION / RAW API

Approach	What Gets Sent	Example	Prompt Tokens (Initial)	Per-Turn Tokens (Avg.)	Notes
 Simple API (cURL / Direct Call) LEANEST	Just the endpoint + minimal auth (if needed)	<code>GET /api/users/123 Authorization: Bearer ...</code>	~120 – 250	~80 – 150	✓ No schema. No tooling overhead. Max payload for reasoning.
 Function Signature (OpenAPI / Function) MODERATE	Function name, description, parameters (JSON Schema)	<code>getUser(id: string): User { id: string (required) ... }</code>	~600 – 1,200	~300 – 600	✓ Smaller than full MCP, but still adds up with many tools.
 MCP Tool (JSON Schema via Manifest) HEAVIEST	Full tool object: name, description, inputSchema, examples, etc.	<code>{ "name": "get_user", "description": "Gets a user...", "inputSchema": { ... }, "examples": [...] }</code>	~1,800 – 3,500	~800 – 1,800	✗ Tool richness comes at a significant token cost.

i Numbers are approximations (GPT-4o tokenizer). Actuals vary by schema size and descriptions. More tools = more tokens. Multiply by turns.

3. STRATEGIES FOR TOKEN-LEAN MCP DESIGN

 Keep Tools Focused One clear responsibility per tool. Avoid kitchen-sink tools.	 Minimize Descriptions Be precise and concise. Cut repetition and unnecessary examples.	 Smart Discovery Expose search/lookup instead of returning big lists.	 On-Demand Loading Load tools only when relevant, not all at session start.	 Prefer Simple APIs If a cURL works, an MCP server may be overkill. Stay pragmatic.
✓ Design for clarity and minimal surface area, not maximum expressiveness.				

i Rule of Thumb: If the tool saves < 2-3 turns of reasoning, a simple API call is usually the better choice.

2. HIDDEN TOKEN COSTS THAT ADD UP

 Instructional Prompts Inside MCP Long instructions, policies, examples, and security guidance in the server.	+200 – 1,000 tokens per tool
 Multi-step “Agentic Loops” Each thinking step = new messages, tool calls, observations, and reasoning.	+400 – 1,500 tokens per loop
 Resource Discovery (e.g., list_*, search_*) Large lists returned and re-sent in context inflate history quickly.	+500 – 2,000+ tokens per discovery
 Long Conversation History Every past tool call, result, and thought stays in the window.	+N × turns tokens (linear growth)
⚠ It's not just the tool. It's the instructions, the loops, the history.	

4. TOKEN IMPACT EXAMPLE (10-TOOL WORKFLOW, 5 TURNS)

Approach	Initial Load Tokens	Per-Turn (Avg.)	5 Turns Total (Est.)	Context Window Used (Est.)
Simple API Calls	~250	~120	~850	~2–4% of 32K
Function Signatures	~900	~450	~3,150	~8–12% of 32K
MCP (10 Tools)	~2,400	~1,100	~7,900	~20–25% of 32K



Save tokens, save money, get better reasoning.
Pick the right interface for the right job.



Costs vary by model and pricing. Use this data to make architectural trade-offs.

Worth It Framework



MCP
Dev Summit
Seoul



5-Point “Painless” Checklist



If you can't check **at least three**, you're likely over-engineering.
This is your rubric for the next sprint planning.

1 Distribution 2+ clients?



Will two or more distinct clients (agents, apps, or teams) use this capability?

Check:
☒ YES ☐ NO

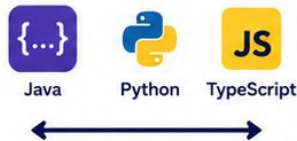
2 Discovery Hierarchical data?



Does the client need to explore, search, or navigate unknown or evolving hierarchical data?

Check:
☒ YES ☐ NO

3 Interop Cross-language / Cross-team?



Does this need to work cleanly across languages, frameworks, or team boundaries?

Check:
☒ YES ☐ NO

4 Context High-density info?



Does the client need rich context (bundle of data, relationships, metadata) to do its job well?

Check:
☒ YES ☐ NO

5 Stability Is the API schema settled?



Is the API contract stable enough that we won't be breaking clients frequently?

Check:
☒ YES ☐ NO



3 or more checked
MCP is likely a good fit.
Invest in the server.



1–2 checked
Evaluate carefully.
Start simple. Revisit if needs grow.



0 checked
Don't build an MCP server.
Use a simple API, script, or existing tool instead.



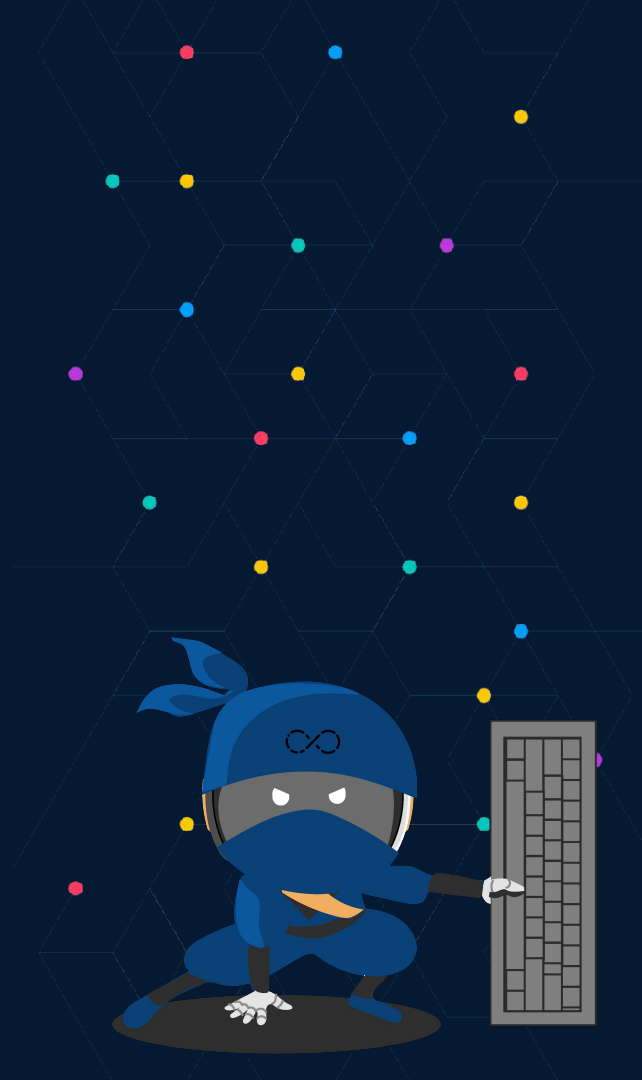
Pro Tip: Re-check this checklist as requirements evolve. What doesn't justify MCP today might tomorrow—but avoid building for a future that isn't real yet.



Live Demo: Over-Engineered Helpdesk



MCP
Dev Summit
Seoul



DEMO SCENARIO



The “Over-Engineered” Helpdesk Server

We built an MCP server to access a helpdesk system.
Let’s see when it helps, and when it doesn’t.

TECH STACK



Quarkus



LangChain4j



MCP SDK



OpenAI

THE COMPARISON: MCP vs. CLEAN SDK (Function Call)

	A. OVER-ENGINEERED MCP SERVER	VS.	B. CLEAN SDK (Direct Function)
Architecture	LLM → MCP (JSON-RPC) → REST → DB		LLM → SDK Function → REST → DB
Tools Available	50+ tools exposed		1 focused function
Prompt Tokens (avg)	~18,400 tokens		~1,250 tokens
Completion Tokens (avg)	~1,320 tokens		~280 tokens
Total Tokens (avg)	~19,720 tokens		~1,530 tokens
Latency (avg)	3.8 – 5.2 sec		0.9 – 1.4 sec
Success Rate	82%		97%
Cost per 100 requests*	~\$2.11		~\$0.16

*Based on gpt-4o pricing (input+output). Your results may vary.

THE PROMPT WE’LL USE



“List all tickets and tell me which one sounds most urgent.”

GITHUB REPO (SAMPLE CODE)



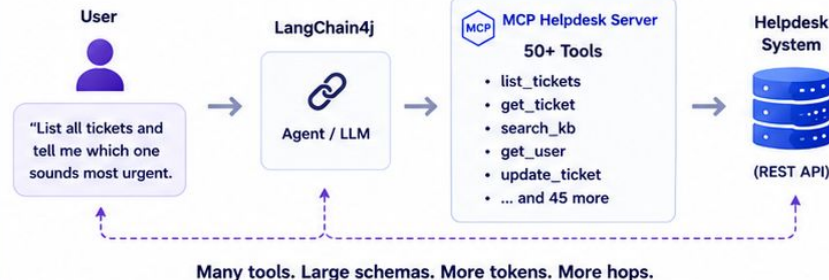
[danieloh30/over-engineered-tool](https://github.com/danieloh30/over-engineered-tool)

This repo contains both:

- Over-engineered MCP server
- Clean SDK (function call) version



OVERVIEW: OUR OVER-ENGINEERED SETUP



THE “AHA!” MOMENT: WHEN MCP PAYS FOR ITSELF

MCP shines when you truly need:

1. MULTI-CLIENT DISTRIBUTION



Multiple apps, agents, or teams consume the same toolset.

2. DYNAMIC DISCOVERY



Agents must discover capabilities at runtime and decide what to use.

3. EVOLVING TOOL ECOSYSTEM



Tools change often, many producers, versioned independently.



Result: Standardization, discoverability, and interoperability outweigh the protocol tax.

SETTING EXPECTATIONS: LIVE CODE



Live demos are like support tickets: unpredictable by nature.



We’ll handle failures together. That’s part of the real-world story.



Goal: Learn, compare, and leave with actionable takeaways.

LIVE CODE



Anything can happen. That’s why we build resilient systems.

Conclusion & The Pragmatic Path Forward

Use MCP where it creates real leverage—avoid it where simplicity wins.



1



The Golden Rule

Only build an MCP server when the **distribution** (multi-client) and **discovery** (autonomous agents) benefits outweigh the protocol tax.



Distribution
(Multi-Client)



Discovery
(Autonomous Agents)



Protocol Tax
(Cost & Complexity)

2



Architect for Agility

Keep your business logic in clean, **protocol-agnostic Java services**; treat MCP as a “disposable” interface layer.



Business Logic
Java Services



MCP Interface Layer
(Disposable)



Clients / Agents

3



Standardize with Intent

Don't let the “Standard” label fool you into over-engineering; a **10-line SDK** is often superior to a **500-line JSON-RPC wrapper**.

10-line SDK (Good)

```
client.callTool("get_user", {
  userId: "123"
});
```



VS.

500-line JSON-RPC Wrapper (Overkill)

```
{ "jsonrpc": "2.0", "id": 1,
  "method": "tools/call",
  "params": { ... 40+ lines ... } }
```



4



Focus on the “Why”

Ask yourself if the LLM actually needs a **dynamic tool registry**, or if it just needs a **well-documented API endpoint**.



Needs Dynamic Tool Registry?

(Autonomous discovery, many evolving tools)

✓ Use MCP

OR



Just Needs an API?

(Known endpoint, stable contract)

✓ Use a Simple API

5



Final Call to Action

Audit your current “Agentic” projects today—identify one MCP server that should have been a simple function call and **refactor** it.



1. Audit
Review your MCP
servers & tools



2. Identify
Find one that could
be a simple API call



3. Refactor
Replace with a simple
function/API



4. Measure
Lower tokens, latency,
and complexity

Ship Faster.
Spend Less.
Keep It Simple.



MCP is a powerful standard—use it where it **multiplies capability**, not where it adds unnecessary complexity.



Simplicity scales.

THANK YOU! QUESTION?

IBM Bob Free Trial



MCP
Dev Summit
Seoul



MCP
Dev Summit
Seoul

